

Module 3

Robotics Math: SE(3), Quaternion, FK/IK

Tài liệu đào tạo nội bộ của BA x AILAB@UEL

Hạng mục	Thông tin
Đối tượng	CTV Phòng BA x AILAB@UEL chưa cần nền tảng robotics sâu
Thời lượng	2 ngày học tập trung + 1 buổi hoàn thiện mini-project
Vai trò trong chương trình	Nền tảng bắt buộc cho Module 4 Camera/RGB-D, Module 6 Motion Primitives và toàn bộ pipeline perception-to-action
Đầu ra	Notebook transform, unit tests, FK/IK mini lab, workspace map, checklist lỗi thường gặp
Phiên bản	v1.2 FULL - giữ nguyên nội dung bản đầu, bổ sung baseline demo và lab output đo được

Thông điệp chính: Module này không nhằm biến sinh viên thành chuyên gia cơ học robot trong 2 ngày. Mục tiêu là đủ hiểu và đủ code để không nhầm hệ tọa độ, không đảo chiều transform, biết đọc pose của vật/robot, biết kiểm tra FK/IK và biết debug lỗi toán học trước khi chạy trong Isaac Sim.

Mục lục

- 1. Vai trò của Module 3 trong HRC2026
- 2. Chuẩn đầu ra và lịch học 2 ngày
- 3. Ôn nhanh toán nền: vector, ma trận, đơn vị
- 4. Frames và quy ước ký hiệu
- 5. Rotations: $SO(3)$, Euler angles, quaternions
- 6. Homogeneous transforms và $SE(3)$
- 7. Forward kinematics: từ joint state đến pose end-effector
- 8. Jacobian, velocity kinematics và singularity
- 9. Inverse kinematics: analytic, numerical, damped least squares
- 10. Workspace, joint limits và collision intuition
- 11. Kết nối với pipeline HRC2026
- 12. Debug checklist và lỗi thường gặp
- 13. Lab thực hành và code skeleton
- 14. Mini-project cuối module và rubric
- 15. Câu hỏi tự kiểm tra
- 16. Tài liệu tham khảo và video tutorial
- 17. Phần bổ sung v1.2: cấu trúc học theo 3 lớp
- 18. Demo trên baseline HRC2026
- 19. Lab bổ sung có output đo được
- 20. Rubric cập nhật và gói nộp Module 3

1. Vai trò của Module 3 trong HRC2026

Theo outline chương trình đã duyệt, Module 3 tập trung vào frames, transforms, kinematics; sinh viên cần biết chuyển đổi quaternion với ma trận quay, tính transform camera $\rightarrow$ world $\rightarrow$ robot base, tính forward/inverse kinematics và trực giác workspace/singularity/collision. Đây là cầu nối giữa Module 2 về Isaac Sim/Walker S2 và các module sau về camera, perception, motion primitives và task integration.

Bài toán trong cuộc thi	Kiến thức Module 3 cần dùng	Sai lầm phổ biến nếu thiếu kiến thức này
Task 1 - Desktop sorting	Đổi pose vật từ camera/world sang robot base; tạo pre-grasp pose; kiểm tra pose đặt vào bin	Robot gấp lệch vài cm do đảo transform hoặc sai trục z
Task 2 - Conveyor sorting	Theo dõi vị trí vật theo thời gian trong cùng frame; dự đoán điểm intercept; kiểm tra workspace	Tay đuổi theo vật đã quá vùng với tới hoặc dự đoán sai hướng
Task 3 - Workpiece insertion	Căn orientation vật với slot foam; kiểm tra pose chèn; hạ theo trục chính xác	Vật bị nghiêng, chạm mép foam, release sai thời điểm
Task 4 - Carton sealing	Mô tả pose hai tay tương đối với nắp carton; hiểu dual-arm frames	Hai tay không đồng bộ, một tay cản tay kia, sequence khó debug

Trong cuộc thi, thân robot được cố định, robot chủ yếu dùng waist và hai cánh tay để thao tác. Vì vậy các lỗi về hệ tọa độ, pose mục tiêu, IK và workspace thường tác động trực tiếp đến success rate, kể cả khi perception hoặc model học máy có vẻ tốt.

2. Chuẩn đầu ra và lịch học 2 ngày

Mã	Sau module, sinh viên có thể...	Minh chứng	
M3-CLO1	Giải thích được frame, point, vector, pose, transform; phân biệt biểu diễn tọa độ và vật thể thật	Trả lời đúng self-check 1-5	
M3-CLO2	Dùng rotation matrix, Euler/RPY và quaternion đúng quy ước; biết rủi ro gimbal lock và thứ tự quaternion	Notebook chuyển đổi rotation + unit tests	
M3-CLO3	Tạo, nhân, nghịch đảo homogeneous transform T in SE(3)	Code utils compose/invert/apply_transform	
M3-CLO4	Tính chuỗi transform camera -> world -> robot base hoặc ngược lại	Bài tập transform chain	
M3-CLO5	Tính FK cho robot 2R planar và hiểu ý nghĩa FK cho arm thật	FK notebook + plot	
M3-CLO6	Tính Jacobian, phát hiện gần singularity bằng determinant/condition number	Jacobian lab	
M3-CLO7	Giải IK analytic cho 2R planar và numerical IK bằng pseudoinverse/DLS	IK lab + convergence log	
M3-CLO8	Vẽ workspace map, kiểm tra joint limits và unreachable targets	Workspace plot + report	
Buổi	Nội dung	Thực hành	Đầu ra
Ngày 1 - Sáng	Vector, frame, rotation matrix, Euler, quaternion	Chuyển đổi RPY <=> quaternion <=> matrix bằng SciPy	rotation_tests.py
Ngày 1 - Chiều	SE(3), transform chain, convention T _{ab}	Tạo/invert/compose T; đổi object pose camera -> base	transform_utils.py + tests
Ngày 2 - Sáng	FK, Jacobian, singularity	2R planar arm FK/Jacobian; workspace sampling	fk_jacobian_lab.ipynb
Ngày 2 - Chiều	IK analytic/numerical, DLS, joint limits	Giải IK cho targets; log hội tụ/thất bại	ik_lab.ipynb
Sau module	Mini-project	Tích hợp toàn bộ vào 1 notebook và báo cáo ngắn	module3_miniproject.zip

3. Ôn nhanh toán nền: vector, ma trận, đơn vị

Robot thao tác trong không gian 3D. Để điều khiển tay robot, ta cần mô tả vị trí, hướng, vận tốc và quan hệ giữa các hệ tọa độ. Phần này nhắc lại các khái niệm tối thiểu, đủ để sinh viên đọc code và debug.

3.1. Point, vector, direction và pose

Khái niệm	Ý nghĩa	Ví dụ
Point	Một điểm có vị trí phụ thuộc vào frame biểu diễn	Tâm vật trong camera frame: $p_c = [0.12, -0.04, 0.75]$ m
Vector	Độ dài hoặc hướng; có thể biểu diễn trong frame cụ thể	Trục tiếp cận gripper: $a_b = [0, 0, -1]$
Frame	Một gốc tọa độ + 3 trục trực giao	world, robot_base, camera, gripper, object
Pose	Vị trí + hướng của một frame/vật thể	Pose object trong base frame: position + quaternion
Transform	Quy tắc đổi biểu diễn từ frame này sang frame khác	T _{base_camera}

3.2. Các phép toán phải dùng thành thạo

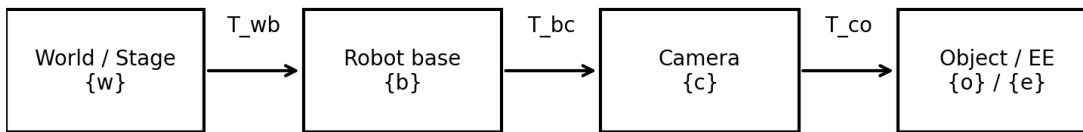
- Norm: $\|v\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$. Dùng để đo độ dài vector hoặc sai số vị trí.

- Normalize: $\hat{v} = v / \|v\|$. Dùng để tạo unit direction; phải kiểm tra $\|v\|$ khác 0.
- Dot product: $a \cdot b = \|a\| \|b\| \cos(\theta)$. Dùng để đo góc tương đối giữa hai hướng.
- Cross product: $a \times b$ tạo vector vuông góc theo quy tắc bàn tay phải.
- Matrix multiplication không giao hoán: $A @ B$ thường khác $B @ A$. Đây là nguồn lỗi lớn trong transform chain.
- Đơn vị: vị trí dùng meter, góc dùng radian trong hầu hết thư viện tính toán; độ chỉ dùng khi đọc/hiển thị.

Quy tắc vàng: Trước khi chạy robot, hãy in shape, đơn vị, frame và convention của mọi biến pose/transform. Một biến tên pose mà không biết đang ở frame nào là một bug tiềm ẩn.

4. Frames và quy ước ký hiệu

Một frame là một hệ tọa độ gồm gốc O và ba trục x, y, z. Trong robotics, cùng một điểm vật lý có thể có nhiều tọa độ số khác nhau tùy frame biểu diễn. Vì vậy quy ước ký hiệu phải nhất quán ngay từ đầu.



Quy tắc: T_{ab} biến tọa độ từ frame $\{b\}$ sang frame $\{a\}$.
 Ví dụ: $p_b = T_{bc} * p_c$; $p_w = T_{wb} * T_{bc} * p_c$.

Hình 1. Chuỗi frame tối thiểu trong pipeline perception-to-action.

4.1. Quy ước T_{ab}

Trong tài liệu này dùng quy ước: T_{ab} là transform đổi tọa độ từ frame $\{b\}$ sang frame $\{a\}$. Nếu p_b là tọa độ điểm p trong frame $\{b\}$, thì:

```
p_a_h = T_ab @ p_b_h
# p_h là homogeneous point [x, y, z, 1]^T
```

Với quy ước này, chuỗi transform được đọc từ phải sang trái:

```
p_world = T_world_base @ T_base_camera @ p_camera
```

Nếu chỉ có T_{ab} mà cần đổi ngược từ $\{a\}$ về $\{b\}$, dùng inverse:

```
T_ba = inv(T_ab)
p_b = T_ba @ p_a
```

4.2. Bảng tên frame khuyến nghị cho HRC2026

Frame	Tên biến gợi ý	Ý nghĩa	Nguồn dữ liệu
world/stage	T_{world_*}	Frame chung của scene trong Isaac Sim	Simulation stage
robot base	T_{base_*}	Frame gắn với base/waist của robot	Robot state / articulation
camera	T_{camera_*}	Frame cảm biến RGB-D	Camera extrinsic
object	T_{object}	Frame gắn với vật cần gấp/chèn	Perception/pose estimator
gripper/end-effector	T_{ee_*}	Frame tại đầu kẹp/tay robot	FK/robot controller
target/pregrasp	T_{target}	Pose mục tiêu cho motion primitive	Planner

5. Rotations: SO(3), Euler angles, quaternions

Hướng của vật hoặc end-effector trong 3D có thể biểu diễn bằng nhiều cách. Không có biểu diễn nào hoàn hảo cho mọi tình huống. Trong code robot, cần biết chuyển đổi an toàn giữa rotation matrix, Euler/RPY và quaternion.

5.1. Rotation matrix và SO(3)

Rotation matrix R là ma trận 3×3 biểu diễn hướng. Một ma trận quay hợp lệ thuộc $SO(3)$, nghĩa là các cột/trục trực giao, có độ dài 1 và $\det(R)=1$.

```
# Điều kiện kiểm tra R hợp lệ
R.T @ R == I
det(R) == 1
```

Ưu điểm	Nhược điểm	Dùng khi nào
Dễ nhân chuỗi transform; trực quan theo trục x,y,z; không có singularity biểu diễn	Dùng 9 số cho 3 bậc tự do; dễ drift do lỗi số; cần re-orthonormalize	SE(3), FK, transform chain, debug trực

5.2. Euler angles / Roll-Pitch-Yaw

Euler/RPY dùng 3 góc nên dễ đọc với con người. Tuy nhiên thứ tự quay rất quan trọng: XYZ, ZYX, intrinsic/extrinsic có thể cho kết quả khác nhau. Euler cũng có rủi ro gimbal lock, tức mất một bậc tự do tại một số cấu hình.

Không dùng Euler làm chuẩn nội bộ: Euler/RPY nên dùng để nhập/hiển thị/debug. Trong API nội bộ, ưu tiên rotation matrix hoặc quaternion, kèm ghi rõ convention.

5.3. Quaternion

Unit quaternion dùng 4 số để biểu diễn orientation. Quaternion tránh gimbal lock và phù hợp nội suy orientation. Tuy nhiên phải chú ý thứ tự thành phần và chuẩn hóa.

Vấn đề	Giải thích	Cách phòng tránh
Thứ tự thành phần	Có thư viện dùng $[x,y,z,w]$, có thư viện dùng $[w,x,y,z]$	Ghi rõ convention trong tên biến hoặc comment; SciPy mặc định scalar-last $[x,y,z,w]$
q và $-q$	Hai quaternion đối dấu biểu diễn cùng một rotation	So sánh rotation qua matrix hoặc dot absolute, không so sánh trực tiếp từng phần tử
Không chuẩn hóa	Sai số số học làm $\ q\ $ khác 1	Normalize trước khi dùng trong controller hoặc convert
Interpolation	Nội suy tuyến tính quaternion có thể sai	Dùng Slerp khi cần mượt orientation

Code mẫu 1. Chuyển đổi Euler/quaternion/rotation matrix với SciPy

```
import numpy as np
from scipy.spatial.transform import Rotation as R

# SciPy dùng quaternion scalar-last: [x, y, z, w]
r = R.from_euler('zyx', [90, 0, 0], degrees=True)
quat_xyzw = r.as_quat()
R_mat = r.as_matrix()
euler_zyx_deg = r.as_euler('zyx', degrees=True)
print(quat_xyzw)
print(R_mat)
print(euler_zyx_deg)
```

5.4. Lab 3.1 - Rotation conversion tests

- Tạo 10 orientation ngẫu nhiên bằng SciPy Rotation.
- Chuyển orientation sang quaternion, rotation matrix, Euler ZYX; chuyển ngược về matrix.
- Viết unit test kiểm tra $R.T @ R$ gần I và $\det(R)$ gần 1.
- Tạo cặp q và $-q$, chứng minh hai quaternion cho cùng một rotation matrix.
- Ghi lại một lỗi do nhầm $[x,y,z,w]$ và $[w,x,y,z]$.

6. Homogeneous transforms và SE(3)

Trong robot manipulation, pose đầy đủ của một vật gồm vị trí p và orientation R . Homogeneous transform gom hai thành phần này vào một ma trận 4×4 để dễ nhân chuỗi, đổi frame và tính inverse.

Homogeneous transform T in SE(3)

R (3x3) orientation	p (3x1)
$0 \ 0 \ 0$	1

$$p_a = T_{ab} * p_{b_h}; \quad T_{ab}^{-1} = [R^T, -R^T p; 0, 1]$$

Hình 2. Cấu trúc block của homogeneous transform.

6.1. Định nghĩa

$$T_{ab} = \begin{bmatrix} R_{ab} & p_{ab} \\ 0 & 1 \end{bmatrix}$$

$$p_{a_h} = T_{ab} @ [x_b, y_b, z_b, 1]^T$$

$$v_a = R_{ab} @ v_b \quad \# \text{ vector hướng không có translation}$$

Điểm point dùng homogeneous coordinate với phần tử cuối là 1, còn direction/vector thuần dùng phần tử cuối là 0 nếu đưa vào ma trận 4x4. Đây là lý do translation tác động lên point nhưng không tác động lên direction.

6.2. Composition và inverse

Composition

$$T_{ac} = T_{ab} @ T_{bc}$$

Inverse của $T_{ab} = [R, p; 0, 1]$

$$T_{ba} = \begin{bmatrix} R^T & -R^T @ p \\ 0 & 1 \end{bmatrix}$$

6.3. Code utilities nên viết trong nhóm**Code mẫu 2. SE(3) helper functions**

```
import numpy as np

def make_T(R_mat, p):
    T = np.eye(4)
    T[:3, :3] = np.asarray(R_mat, dtype=float)
    T[:3, 3] = np.asarray(p, dtype=float).reshape(3)
    return T

def inv_T(T):
    R_mat = T[:3, :3]
    p = T[:3, 3]
    T_inv = np.eye(4)
    T_inv[:3, :3] = R_mat.T
    T_inv[:3, 3] = -R_mat.T @ p
```

```

return T_inv

def transform_point(T, p):
    p_h = np.r_[np.asarray(p).reshape(3), 1.0]
    return (T @ p_h)[:3]

def transform_vector(T, v):
    v_h = np.r_[np.asarray(v).reshape(3), 0.0]
    return (T @ v_h)[:3]

def is_rotation_matrix(R_mat, atol=1e-6):
    return (np.allclose(R_mat.T @ R_mat, np.eye(3), atol=atol)
            and np.isclose(np.linalg.det(R_mat), 1.0, atol=atol))

```

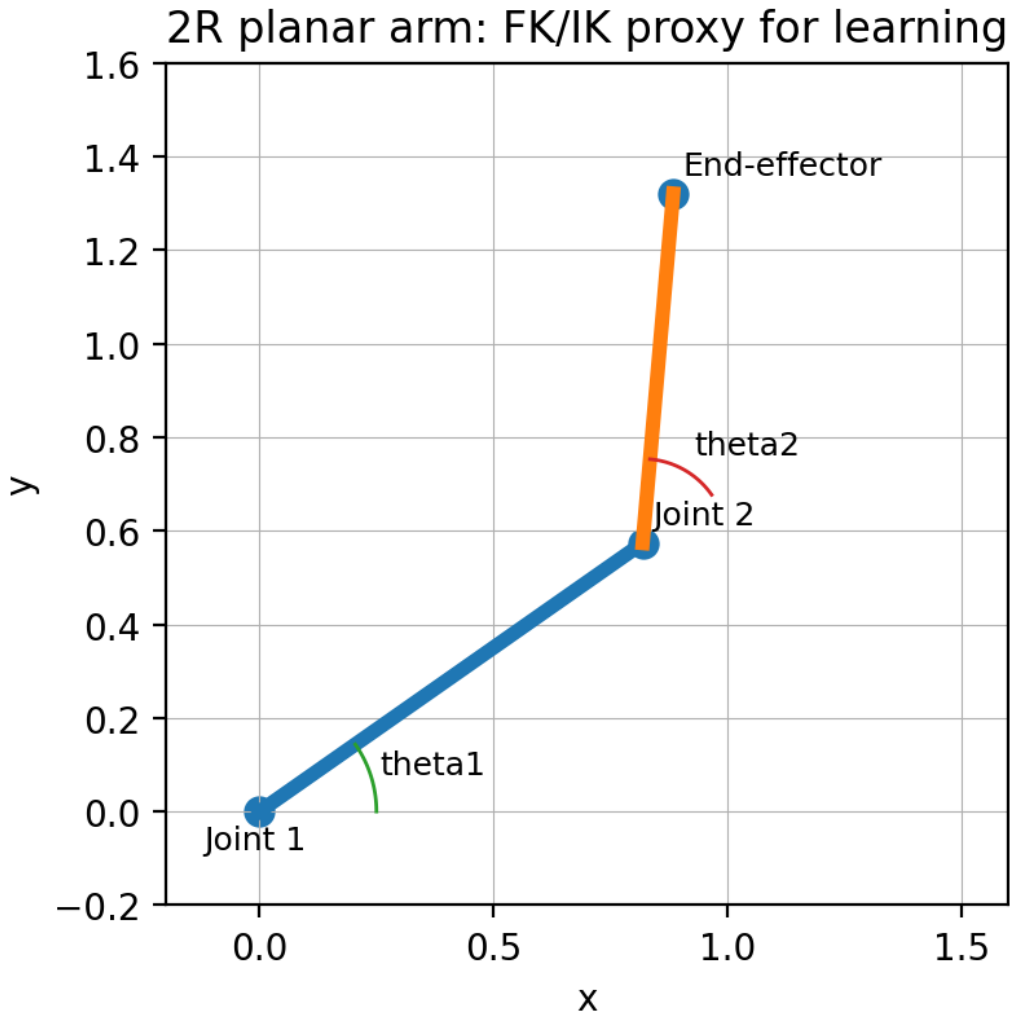
6.4. Lab 3.2 - Transform chain camera -> base

6. Giả sử có $T_{\text{base_camera}}$ và một điểm tâm vật $p_{\text{camera}} = [0.30, -0.08, 0.75]$ m. Tính p_{base} .
7. Tạo một orientation object trong camera frame, chuyển thành $T_{\text{camera_object}}$.
8. Tính $T_{\text{base_object}} = T_{\text{base_camera}} @ T_{\text{camera_object}}$.
9. Kiểm tra $\text{inv}_T(T_{\text{base_camera}}) @ T_{\text{base_camera}}$ gần identity.
10. Viết test chứng minh point chịu translation nhưng direction thì không.

Lỗi khó phát hiện: Nếu dùng nhầm $T_{\text{camera_base}}$ thay vì $T_{\text{base_camera}}$, kết quả vẫn là một vector 3D có vẻ hợp lệ nhưng robot sẽ đi sai hướng. Vì vậy mọi hàm phải ghi rõ input/output frame trong docstring.

7. Forward kinematics: từ joint state đến pose end-effector

Forward kinematics (FK) trả lời câu hỏi: nếu robot có joint values q , end-effector đang ở pose nào trong robot base/world frame? Đây là phép tính nền tảng để kiểm tra robot đang ở đâu trước khi điều khiển đi tới pose mới.



Hình 3. Robot 2R planar dùng làm proxy để học FK/IK trước khi áp dụng vào tay robot thật.

7.1. Ví dụ 2R planar arm

Với hai link $L1$, $L2$ và hai joint θ_1 , θ_2 , vị trí end-effector là:

$$\begin{aligned} x &= L1 \cdot \cos(\theta_1) + L2 \cdot \cos(\theta_1 + \theta_2) \\ y &= L1 \cdot \sin(\theta_1) + L2 \cdot \sin(\theta_1 + \theta_2) \\ \phi &= \theta_1 + \theta_2 \end{aligned}$$

Code mẫu 3. FK cho 2R planar

```
import numpy as np

def fk_2r(theta, L1=1.0, L2=0.75):
    t1, t2 = theta
    x = L1*np.cos(t1) + L2*np.cos(t1 + t2)
    y = L1*np.sin(t1) + L2*np.sin(t1 + t2)
    phi = t1 + t2
    return np.array([x, y, phi])
```

7.2. FK cho serial robot 3D

Với robot arm thật, FK có thể được tính từ URDF, Denavit-Hartenberg parameters hoặc Product of Exponentials (POE). Trong HRC2026, sinh viên không cần tự derive toàn bộ FK Walker S2 trong 2 ngày; mục tiêu là hiểu input/output và biết kiểm tra kết quả từ thư viện/baseline.

Cách mô tả	Ý tưởng	Khi nào dùng
URDF / robot description	Mô tả link, joint, origin, axis; phần mềm tính transform chain	Khi đọc model robot từ baseline/Isaac/ROS
DH parameters	Mỗi joint/link có a, alpha, d, theta	Robot arm cổ điển, học thuật

Product of Exponentials	Dùng screw axes và home configuration M	Modern Robotics, kiểm tra FK/IK cấp cao
-------------------------	---	---

Code mẫu 4. FK với Modern Robotics package

```
# Ví dụ dùng package modern_robotics sau khi cài: pip install modern-robotics
# Lưu ý: ví dụ mang tính học tập, không phải tham số Walker S2.
import modern_robotics as mr
import numpy as np

M = np.array([[ -1, 0, 0, 0],
               [ 0, 1, 0, 6],
               [ 0, 0, -1, 2],
               [ 0, 0, 0, 1]])
Slist = np.array([[0, 0, 1, 4, 0, 0],
                  [0, 0, 0, 0, 1, 0],
                  [0, 0, -1, -6, 0, -0.1]]).T
thetalist = np.array([np.pi/2, 3, np.pi])
T = mr.FKinSpace(M, Slist, thetalist)
print(T)
```

7.3. Lab 3.3 - FK tests

- Viết $fk_2r(\theta)$ và kiểm thử các cấu hình để biết trước: $\theta = [0, 0], [\pi/2, 0], [0, \pi]$.
- Vẽ 5 cấu hình robot 2R trên cùng một plot.
- Dùng Modern Robotics hoặc SpatialMath để tạo một transform 3D đơn giản và so sánh với hàm `make_T` tự viết.
- Viết docstring cho hàm FK: input q có đơn vị gì, output pose ở frame nào.

8. Jacobian, velocity kinematics và singularity

Jacobian $J(q)$ liên hệ vận tốc joint với vận tốc end-effector. Ở mức trực giác: nếu FK cho biết pose hiện tại, thì Jacobian cho biết pose sẽ thay đổi như thế nào khi joint thay đổi rất nhỏ.

8.1. Ý nghĩa

```
end_effector_velocity = J(q) @ joint_velocity
# Trong 2D: [x_dot, y_dot]^T = J(q) @ [theta1_dot, theta2_dot]^T
# Trong 3D: twist 6D = J(q) @ q_dot
```

Jacobian quan trọng vì IK numerical thường cập nhật joint dựa trên J hoặc pseudo-inverse của J . Khi Jacobian suy giảm hạng hoặc condition number rất lớn, robot ở gần singularity và việc điều khiển có thể trở nên không ổn định.

8.2. Jacobian cho 2R planar**Code mẫu 5. Jacobian 2R và kiểm tra condition number**

```
def jacobian_2r(theta, L1=1.0, L2=0.75):
    t1, t2 = theta
    j11 = -L1*np.sin(t1) - L2*np.sin(t1 + t2)
    j12 = -L2*np.sin(t1 + t2)
    j21 = L1*np.cos(t1) + L2*np.cos(t1 + t2)
    j22 = L2*np.cos(t1 + t2)
    return np.array([[j11, j12],
                     [j21, j22]])

def near_singular(J, threshold=100.0):
    # condition number cao: gần singularity
    return np.linalg.cond(J) > threshold
```

8.3. Singularity là gì?

- Ở singularity, một số hướng chuyển động của end-effector trở nên khó hoặc không thể tạo ra bằng joint velocities hữu hạn.

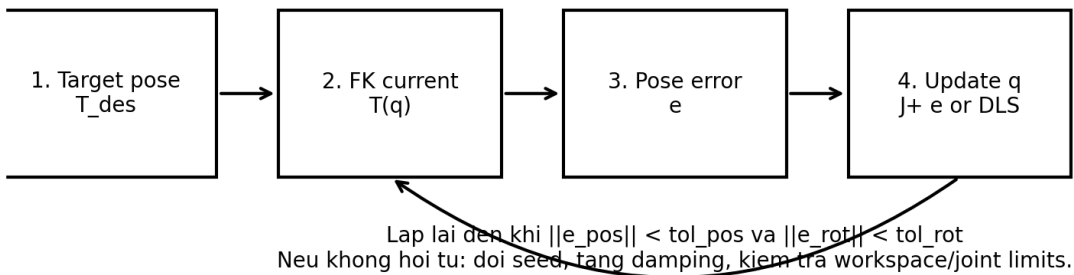
- Một robot arm duỗi thẳng hoàn toàn thường gần singularity trong ví dụ 2R planar.
- IK gần singularity có thể tạo bước joint rất lớn, gây rung, vượt joint limits hoặc collision.
- Damped least squares là một cách làm IK ổn định hơn pseudo-inverse thuần khi gần singularity.

8.4. Lab 3.4 - Jacobian finite-difference check

15. Tính J analytic cho 2R tại $\theta = [0.5, 0.7]$.
16. Tính J numerical bằng finite difference: thay đổi θ_i một epsilon nhỏ và quan sát delta position.
17. So sánh hai kết quả bằng `np.allclose`.
18. Quét θ_2 từ $-\pi$ đến π , vẽ condition number của J; chỉ ra vùng gần singularity.

9. Inverse kinematics: analytic, numerical, damped least squares

Inverse kinematics (IK) trả lời câu hỏi ngược lại FK: muốn end-effector đạt target pose, cần joint values q nào? IK khó hơn FK vì target có thể ngoài workspace, có nhiều nghiệm, hoặc không có nghiệm phù hợp joint limits/collision.



Hình 4. Vòng lặp IK numerical ở mức khái niệm.

9.1. IK analytic cho 2R planar

Với 2R planar, target là vị trí (x, y) . Có thể giải bằng định lý cos. Bên trong workspace thường có hai nghiệm: elbow-up và elbow-down.

Code mẫu 6. IK analytic 2R

```
def ik_2r_analytic(x, y, L1=1.0, L2=0.75, elbow=1):
    r2 = x*x + y*y
    c2 = (r2 - L1*L1 - L2*L2) / (2*L1*L2)
    if c2 < -1.0 or c2 > 1.0:
        return None # unreachable
    s2 = elbow * np.sqrt(max(0.0, 1 - c2*c2))
    t2 = np.arctan2(s2, c2)
    k1 = L1 + L2*c2
    k2 = L2*s2
    t1 = np.arctan2(y, x) - np.arctan2(k2, k1)
    return np.array([t1, t2])
```

9.2. IK numerical bằng pseudo-inverse

Code mẫu 7. IK bằng pseudo-inverse

```
def ik_2r_pinv(target_xy, q0, L1=1.0, L2=0.75, max_iter=100, tol=1e-4, alpha=0.7):
    q = np.array(q0, dtype=float)
```

```

target_xy = np.asarray(target_xy, dtype=float)
history = []
for k in range(max_iter):
    pos = fk_2r(q, L1, L2)[:2]
    e = target_xy - pos
    err = np.linalg.norm(e)
    history.append(err)
    if err < tol:
        return q, True, history
    J = jacobian_2r(q, L1, L2)
    dq = alpha * np.linalg.pinv(J) @ e
    q = q + dq
return q, False, history

```

9.3. Damped least squares

Damped least squares (DLS) giảm độ nhạy khi J gần singularity. Công thức cập nhật phổ biến:

$$dq = J.T @ \text{inv}(J @ J.T + \lambda^2 * I) @ e$$

Code mẫu 8. Một bước DLS

```

def dls_step(J, e, damping=0.05):
    m = J.shape[0]
    return J.T @ np.linalg.solve(J @ J.T + (damping**2) * np.eye(m), e)

```

9.4. Những điều kiện cần kiểm tra khi dùng IK

Điều kiện	Câu hỏi cần hỏi	Hành động
Reachability	Target có nằm trong workspace không?	Nếu không, chọn target gần nhất hoặc bỏ qua vật
Joint limits	Nghiệm q có vượt giới hạn joint không?	Clamp, penalty hoặc đổi seed
Collision	Đường đi có đụng bàn/hộp/foam/robot không?	Thêm pre-grasp/pre-place và kiểm tra collision
Singularity	J có condition number quá lớn không?	Dùng damping, đổi tư thế, tránh duỗi thẳng
Convergence	IK có giảm error qua mỗi vòng không?	Log error, đổi seed/alpha/damping
Frame correctness	Target pose đang ở đúng robot base frame chưa?	Kiểm tra transform chain trước IK

9.5. Lab 3.5 - IK target sweep

- Sinh 50 target ngẫu nhiên trong workspace của 2R planar.
- Giải IK bằng analytic và numerical; so sánh nghiệm bằng FK error.
- Thêm 10 target ngoài workspace và ghi rõ cách phát hiện thất bại.
- Vẽ error history của ít nhất 3 target: dễ, gần singularity, ngoài workspace.

10. Workspace, joint limits và collision intuition

Workspace là tập các vị trí/pose mà end-effector có thể đạt được với joint limits cho trước. Trong HRC2026, workspace quyết định robot có nên gấp vật đó không, gấp bằng tay nào, cần pre-position ra sao, và có nên bỏ qua vật trên băng chuyền hay không.

10.1. Workspace sampling

Code mẫu 9. Sampling workspace 2R

```

def sample_workspace(n=20000, L1=1.0, L2=0.75,
                    q1_lim=(-np.pi, np.pi), q2_lim=(-np.pi, np.pi)):
    qs = np.column_stack([
        np.random.uniform(*q1_lim, size=n),
        np.random.uniform(*q2_lim, size=n)
    ])
    pts = np.array([fk_2r(q, L1, L2)[:2] for q in qs])
    return qs, pts

```

10.2. Collision intuition

- Collision không chỉ xảy ra tại target; đường đi từ pose hiện tại đến target cũng có thể va chạm.

- Pre-grasp/pre-place là pose trung gian an toàn, thường cách object/slot theo trục tiếp cận một khoảng vài cm.
- Motion primitive nên có vùng cấm: bàn, thành hộp, foam, nắp carton, thân robot, cánh tay còn lại.
- Nếu một target nằm rất gần rìa workspace, nên giảm tốc và tăng margin; không nên tối ưu tốc độ trước khi ổn định.

10.3. Lab 3.6 - Workspace map

23. Sampling 10.000 cấu hình 2R với joint limits giả định.
24. Vẽ scatter workspace và đánh dấu các target reachable/unreachable.
25. Tính tỷ lệ target reachable trong một vùng làm việc giả định.
26. Viết hàm choose_reachable_target(targets) chọn target gần nhất và trong workspace.

11. Kết nối với pipeline HRC2026

Module 3 phải tạo ra một 'contract' giữa perception, planning và motion. Các module sau chỉ hoạt động ổn định nếu mỗi pose đều ghi rõ frame, unit và convention.

Module sau	Đầu vào cần từ Module 3	Ví dụ interface
Module 4 Camera/RGB-D	Camera intrinsics/extrinsics; transform point từ camera sang base	object.pose_base = T_base_camera @ object.pose_camera
Module 5 Perception	Pose object chuẩn hóa: position + quaternion + confidence + frame_id	Pose3D(position, quat_xyzw, frame='robot_base')
Module 6 Motion primitives	Pre-grasp/place pose; IK target; workspace check	plan_pick(T_base_object, grasp_axis, hand='right')
Module 10 Evaluation	Metric sai số pose, log transform/IK failure	error_pos_m, error_rot_deg, ik_success

11.1. Pose object nên chuẩn hóa thế nào?

Code mẫu 10. Data class pose có frame_id

```
from dataclasses import dataclass
import numpy as np

@dataclass
class Pose3D:
    position_m: np.ndarray          # shape (3,), meters
    quat_xyzw: np.ndarray          # shape (4,), unit quaternion, scalar-last
    frame_id: str                  # e.g., 'robot_base'
    confidence: float = 1.0

    def validate(self):
        assert self.position_m.shape == (3,)
        assert self.quat_xyzw.shape == (4,)
        assert np.isclose(np.linalg.norm(self.quat_xyzw), 1.0, atol=1e-4)
        assert self.frame_id in {'world', 'robot_base', 'camera', 'gripper'}
```

11.2. Ví dụ HRC-style: tạo pre-grasp pose

Giả sử perception trả về object pose trong robot_base. Planner muốn tạo pre-grasp pose cao hơn vật 8 cm theo trục z của base và giữ cùng orientation của object hoặc orientation gripper đã định nghĩa.

Code mẫu 11. Pre-grasp pose đơn giản

```
def make_pregrasp(T_base_object, approach_offset_m=0.08):
    T_base_pre = T_base_object.copy()
    # Đơn giản: lui theo +z base. Trong robot thật có thể lui theo trục grasp axis.
    T_base_pre[:3, 3] = T_base_object[:3, 3] + np.array([0.0, 0.0, approach_offset_m])
    return T_base_pre
```

12. Debug checklist và lỗi thường gặp

Triệu chứng	Nguyên nhân có thể	Cách debug
-------------	--------------------	------------

Robot đi ngược hướng	Nhầm T _{ab} với T _{ba} ; trục camera khác trục base	In transform direction; kiểm tra inv_T; visualize axes
Object pose lệch cố định vài cm	Sai extrinsic hoặc đơn vị mm/m	Kiểm tra đơn vị; test với điểm gốc; đo offset
Orientation xoay sai 90/180 độ	Nhầm Euler order hoặc trục object/gripper	Vẽ axes; chuyển qua rotation matrix; kiểm tra determinant
Quaternion có vẻ đúng nhưng robot xoay kỳ lạ	Nhầm [x,y,z,w] và [w,x,y,z]	Ghi rõ convention; test conversion SciPy
IK không hội tụ	Target ngoài workspace, seed kém, singularity	Log error history; đổi seed; dùng DLS; kiểm tra workspace
IK tạo joint nhảy lớn	Gần singularity hoặc damping quá thấp	Tăng damping; giới hạn step dq; tránh pose xoắn thẳng
FK test lúc đúng lúc sai	Dùng degree thay vì radian	Chuẩn hóa mọi góc về radian trong code
Unit test pass nhưng robot vẫn lỗi	Frame convention đúng trong test nhưng sai input từ module khác	Thêm frame_id và validation ở boundary module
Checklist trước khi chạy robot: 1) target pose ở robot_base chưa? 2) position là meter chưa? 3) quaternion là [x,y,z,w] và đã normalize chưa? 4) IK có success flag không? 5) target có trong workspace không? 6) có pre-grasp/pre-place chưa? 7) log đủ để replay/debug chưa?		

13. Lab thực hành và code skeleton

Sinh viên nên làm toàn bộ lab trong một notebook `module3\_robotics\_math.ipynb` và tách các hàm dùng lại vào `robot\_math\_utils.py`. Dưới đây là cấu trúc gợi ý.

Cấu trúc thư mục lab

```
module3_miniproject/
  module3_robotics_math.ipynb
  robot_math_utils.py
  tests/
    test_rotation.py
    test_transform.py
    test_fk_ik.py
  outputs/
    workspace_map.png
    ik_error_history.png
  README.md
```

13.1. robot_math_utils.py skeleton

Code skeleton tối thiểu

```
import numpy as np
from scipy.spatial.transform import Rotation as R

# ----- rotation -----
def quat_xyzw_to_R(q):
    q = np.asarray(q, dtype=float).reshape(4)
    q = q / np.linalg.norm(q)
    return R.from_quat(q).as_matrix()

def R_to_quat_xyzw(R_mat):
    return R.from_matrix(R_mat).as_quat()

# ----- SE(3) -----
def make_T(R_mat, p):
    T = np.eye(4)
    T[:3, :3] = R_mat
    T[:3, 3] = np.asarray(p).reshape(3)
    return T

def inv_T(T):
    R_mat, p = T[:3, :3], T[:3, 3]
    out = np.eye(4)
    out[:3, :3] = R_mat.T
```

```

out[:3,3] = -R_mat.T @ p
return out

# ----- 2R FK/IK -----
def fk_2r(q, L1=1.0, L2=0.75):
    t1, t2 = q
    return np.array([
        L1*np.cos(t1) + L2*np.cos(t1+t2),
        L1*np.sin(t1) + L2*np.sin(t1+t2),
        t1+t2
    ])

def jacobian_2r(q, L1=1.0, L2=0.75):
    t1, t2 = q
    return np.array([
        [-L1*np.sin(t1)-L2*np.sin(t1+t2), -L2*np.sin(t1+t2)],
        [ L1*np.cos(t1)+L2*np.cos(t1+t2),  L2*np.cos(t1+t2)]
    ])

```

13.2. Unit test gợi ý

Unit tests gợi ý với pytest

```

def test_transform_inverse_identity():
    R_mat = R.from_euler('zyx', [20, 10, 5], degrees=True).as_matrix()
    T = make_T(R_mat, [0.1, 0.2, 0.3])
    assert np.allclose(inv_T(T) @ T, np.eye(4), atol=1e-8)

def test_fk_2r_zero():
    out = fk_2r([0.0, 0.0], L1=1.0, L2=0.75)
    assert np.allclose(out[:2], [1.75, 0.0], atol=1e-8)

def test_quaternion_sign_equivalence():
    q = R.from_euler('xyz', [15, 20, 30], degrees=True).as_quat()
    assert np.allclose(R.from_quat(q).as_matrix(), R.from_quat(-q).as_matrix())

```

14. Mini-project cuối module

Tên mini-project: `Transform & IK Mini Lab for HRC2026`. Mục tiêu là chứng minh sinh viên có thể tự viết và kiểm tra các phép toán robot math tối thiểu trước khi dùng trong Isaac Sim/baseline.

14.1. Yêu cầu nộp

27. Một notebook `module3\_robotics\_math.ipynb` chạy từ đầu đến cuối không lỗi.
28. Một file `robot\_math\_utils.py` chứa các hàm rotation/SE(3)/FK/IK có docstring rõ frame/unit/convention.
29. Một thư mục `tests/` có ít nhất 6 unit tests bằng pytest hoặc assert trong notebook.
30. Hai hình output: `workspace\_map.png` và `ik\_error\_history.png`.
31. Một `README.md` dài 1-2 trang mô tả kết quả, lỗi gặp phải và cách debug.

14.2. Các nhiệm vụ bắt buộc

Task	Mô tả	Điểm kiểm tra
A. Rotation	Tạo 10 rotations, chuyển Euler/quaternion/matrix qua lại; kiểm tra q và -q	Không sai convention; R hợp lệ
B. SE(3)	Viết make_T, inv_T, transform_point/vector; test inverse và composition	T @ inv_T(T) gần identity
C. Transform chain	Tạo T_base_camera và T_camera_object giả lập; tính T_base_object	Giải thích đúng T_ab
D. FK/Jacobian	Viết FK và Jacobian 2R; check finite difference	Sai số finite difference nhỏ
E. IK	Giải 50 targets bằng analytic/numerical; log success/failure	FK(q_solution) gần target
F. Workspace	Sampling workspace và phân loại target reachable/unreachable	Có plot và nhận xét

14.3. Rubric chấm mini-project

Hạng mục	Điểm	Mô tả đạt điểm tối đa
Rotation/quaternion	15	Chuyển đổi đúng, normalize, ghi rõ $[x, y, z, w]$, có tests
SE(3) utilities	20	make/invert/compose/apply transform đúng; point/vector phân biệt; có tests
Transform chain	15	Tính đúng camera/object/base chain; giải thích convention T_ab
FK/Jacobian	15	FK 2R đúng, Jacobian analytic khớp finite difference
IK	15	Analytic hoặc numerical chạy được; detect unreachable; có convergence log
Workspace map	10	Sampling/plot rõ, nhận xét singularity/reachability
Code quality/reproducibility	10	Notebook chạy sạch, README rõ, code có docstring, không hard-code mở hồ

14.4. Tiêu chí đạt

- Từ 80 điểm trở lên: đủ năng lực vào nhóm Manipulation/Action Planning hoặc hỗ trợ Perception-to-Pose.
- Từ 65-79 điểm: đạt nền tảng, cần mentor review thêm trước khi sửa baseline robot math.
- Dưới 65 điểm: cần học lại frame/SE(3) trước khi làm IK hoặc motion primitive.

15. Câu hỏi tự kiểm tra

- Phân biệt point và vector trong homogeneous coordinate. Vì sao point có phần tử cuối là 1 còn vector là 0?
- Nếu T_{ab} đổi từ frame b sang frame a, công thức đổi p_b sang p_a là gì?
- Vì sao $A @ B$ khác $B @ A$ quan trọng trong transform chain?
- Rotation matrix hợp lệ cần thỏa hai điều kiện nào?
- Tại sao không nên dùng Euler angles làm biểu diễn nội bộ duy nhất?
- SciPy Rotation mặc định quaternion theo thứ tự nào?
- Vì sao q và -q biểu diễn cùng orientation?
- Viết công thức inverse của $T = [R, p; 0, 1]$.
- FK và IK khác nhau như thế nào?
- Vì sao IK có thể không có nghiệm hoặc có nhiều nghiệm?
- Jacobian dùng để làm gì trong IK numerical?
- Damped least squares giúp ích gì khi gần singularity?
- Workspace khác task space như thế nào?
- Hãy nêu ba lỗi thường gặp khi chuyển object pose từ camera sang robot base.
- Trước khi gửi target pose cho controller, cần validate những trường nào?

Gợi ý sử dụng self-check: Sinh viên tự trả lời ngắn 2-4 câu/câu. Mentor chỉ cần kiểm tra các câu 2, 6, 8, 10, 11, 14, 15 để phát hiện sớm lỗi tư duy frame và IK.

16. Tài liệu tham khảo và video tutorial

Ưu tiên đọc/xem theo thứ tự: Modern Robotics Chapter 3 -> Chapter 4 -> Chapter 5 phần Jacobian/Singularity -> Chapter 6 IK -> SciPy Rotation -> ROS tf2. Với sinh viên mới, chỉ cần đọc/xem các phần tương ứng lab, không cần học toàn bộ dynamics/control.

- **Global Humanoid Robot Challenge 2026 - Baseline GitHub** - https://github.com/UBTECH-Robot/GlobalHumanoidRobotChallenge_2026_Baseline. Repo baseline chính thức để liên hệ robot math với pipeline Isaac Sim/LeRobot.
- **HRC2026 Simulation Assets - Hugging Face** - https://huggingface.co/UBTECH-Robotics/challenge2026_assets. Assets Walker S2, URDF/USD, scene elements, task components.
- **Modern Robotics - Chapter 3: Rigid-Body Motions** - <https://modernrobotics.northwestern.edu/chapters/chapter3/>. Video nền tảng về frame, rotation matrices, homogeneous transforms, twists.
- **Modern Robotics - 3.3.1 Homogeneous Transformation Matrices** - <https://modernrobotics.northwestern.edu/nu-gm-book-resource/3-3-1-homogeneous-transformation-matrices/>. Video trực tiếp về SE(3) và ba cách dùng transform matrices.

- **Modern Robotics - Chapter 4: Forward Kinematics -**
<https://modernrobotics.northwestern.edu/chapters/chapter4/>. POE formula, FK trong space frame/end-effector frame, ví dụ FK.
- **Modern Robotics - 4.1.1 Product of Exponentials in the Space Frame -**
<https://modernrobotics.northwestern.edu/nu-gm-book-resource/4-1-1-product-of-exponentials-formula-in-the-space-frame/>. Video FK bằng Product of Exponentials.
- **Modern Robotics - Chapter 5: Velocity Kinematics and Statics -**
<https://modernrobotics.northwestern.edu/chapters/chapter5/>. Jacobian, singularity, manipulability.
- **Modern Robotics - 5.3 Singularities -** <https://modernrobotics.northwestern.edu/nu-gm-book-resource/5-3-singularities/>. Video riêng về singularity.
- **Modern Robotics - Chapter 6: Inverse Kinematics -**
<https://modernrobotics.northwestern.edu/chapters/chapter6/>. IK open chains, numerical IK, Newton-Raphson.
- **Modern Robotics - Inverse Kinematics of Open Chains -**
<https://modernrobotics.northwestern.edu/nu-gm-book-resource/inverse-kinematics-of-open-chains/>. Video IK analytic/numerical và tính nhiều nghiệm/không có nghiệm.
- **NxRLab/ModernRobotics Code Library -** <https://github.com/NxRLab/ModernRobotics>. Code Python/MATLAB/Mathematica dùng cho FK, Jacobian, IK; phù hợp học tập.
- **modern-robotics PyPI -** <https://pypi.org/project/modern-robotics/>. Cài package Python: pip install modern-robotics.
- **ROS 2 tf2 Documentation -** <https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Tf2.html>. Tư duy quản lý nhiều coordinate frames theo thời gian.
- **ROS 2 tf2 Tutorials -** <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>. Tutorial thực hành tf2 bằng C++/Python.
- **SciPy spatial.transform.Rotation -**
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html>. Chuyển đổi quaternion, Euler, rotation matrix, rotvec, apply/invert rotations.
- **SciPy Spatial Transformations -** <https://docs.scipy.org/doc/scipy/reference/spatial.transform.html>. Rotation, Slerp và rigid transformations.
- **SpatialMath Python -** <https://pypi.org/project/spatialmath-python/>. SO3/SE3/UnitQuaternion classes cho robotics/vision.
- **Robotics Toolbox for Python -** <https://petercorke.github.io/robotics-toolbox-python/index.html>. Robot models, FK/IK, trajectories, differential kinematics.
- **Robotics Toolbox Python - Kinematics Wiki -** <https://github.com/petercorke/robotics-toolbox-python/wiki/Kinematics>. Ví dụ fkin, ikin và các solver IK.
- **YouTube - Modern Robotics 3.3.1 Homogeneous Transformation Matrices -**
<https://www.youtube.com/watch?v=vIb3P7arbkU>. Video tutorial trực tiếp trên YouTube.
- **YouTube - Modern Robotics Chapter 3 Introduction to Rigid-Body Motions -**
<https://www.youtube.com/watch?v=29LhXWjn7Pc>. Video nhập môn về body frame, space frame, quy tắc bàn tay phải.

Phụ lục A. Cheat sheet công thức

Chủ đề	Công thức / Quy tắc
Rotation matrix	$R.T @ R = I$; $\det(R)=1$
Quaternion unit	$\ q\ =1$; q và $-q$ cùng orientation; SciPy: $[x,y,z,w]$
SE(3)	$T = [R \ p; 0 \ 1]$
Transform point	$p_a _h = T_ab @ p_b _h$
Transform vector	$v_a = R_ab @ v_b$ hoặc $[v;0]$
Inverse transform	$T_ba = [R_ab.T, -R_ab.T @ p_ab; 0, 1]$
Composition	$T_ac = T_ab @ T_bc$
FK	$q \rightarrow T_base _ee$
Jacobian	$x_dot = J(q) @ q_dot$
Pseudo-inverse IK	$dq = J^+ @ e$
DLS IK	$dq = J.T @ \text{inv}(J @ J.T + \lambda^2 I) @ e$
2R reachability	$ L1-L2 \leq \sqrt{x^2+y^2} \leq L1+L2$

17. Phần bổ sung v1.2: cấu trúc học theo 3 lớp

Mục tiêu của phần bổ sung. Phần này không thay thế các nội dung lý thuyết đã có trong Module 3 bản ban đầu. Toàn bộ các mục về vector, frame, rotation matrix, Euler angle, quaternion, SE(3), FK, Jacobian, IK, workspace, singularity, collision intuition, debug checklist, lab skeleton và tài liệu tham khảo vẫn được giữ nguyên. Phần bổ sung chỉ làm rõ cách sinh viên chuyển từ lý thuyết sang baseline HRC2026 và cách tạo sản phẩm lab có thể đo, chấm, tái lập được.

Lớp học	Câu hỏi sinh viên phải trả lời được	Bảng chứng cần nộp
Lý thuyết tối thiểu	Frame nào đang được dùng? Pose gồm vị trí và hướng gì? Transform T_A_B đổi tọa độ theo chiều nào? FK/IK trả lời câu hỏi gì?	Ghi chú công thức, unit tests, giải thích convention trong README.
Demo trên baseline	Khái niệm toán học này xuất hiện ở file nào trong baseline/assets? Joint/link/end-effector/camera/pose được đọc ở đâu?	baseline_math_map.md, joint_inventory.csv, frame_inventory.csv, ảnh/log demo.
Lab output đo được	Kết quả có đúng không? Sai số bao nhiêu? Test có pass không? Có tái lập được trên máy khác không?	pytest_report.txt, rotation_report.csv, transform_chain_report.json, ik_2r_report.csv, workspace.png.

Nguyên tắc chấm: không chấm theo việc sinh viên “hiểu miệng” mà chấm theo output chạy được: file, log, test, hình, bảng sai số và README giải thích được convention.

18. Demo trên baseline HRC2026

Mục tiêu. Sinh viên nhìn thấy các khái niệm toán học của Module 3 trong repo baseline, assets, URDF/USD và config task. Demo chia thành hai mức: file-level demo có thể chạy ngay cả khi chưa mở được Isaac Sim GUI; simulation-level demo dùng khi môi trường Module 1-2 đã sẵn sàng.

Demo	Chạy khi nào?	Mục tiêu	Output đo được
Demo 0 - Baseline math map	Sau khi clone baseline	Tìm các file có từ khóa pose, transform, quaternion, ik, jacobian, joint, gripper, camera.	outputs/baseline_math_map.md
Demo 1 - URDF joint/link inventory	Sau khi tải Walker S2 assets	Liệt kê link, joint, parent/child, joint type, axis, limit.	outputs/joint_inventory.csv
Demo 2 - Frame/config inventory	Sau khi có YAML/USD/config	Tìm camera, robot base, world, end-effector, object/bin/foam/carton frame nếu có.	outputs/frame_inventory.csv
Demo 3 - Transform sanity	Khi chạy được Python trong env	Kiểm tra $T @ \text{inv}(T)$ xấp xỉ I, quaternion round-trip, convention xyzw/wxyz.	outputs/transform_sanity_report.json
Demo 4 - IK dry-run	Khi baseline/IK API sẵn sàng	Gửi target pose an toàn, kiểm tra solver trả joint hợp lệ, residual thấp, không vượt joint limit.	outputs/ik_dryrun_report.csv

18.1. Demo 0 - Tạo baseline math map

Ý nghĩa. Trước khi sửa code điều khiển, sinh viên phải biết trong baseline các khái niệm pose/transform/IK nằm ở đâu. Demo này dùng tìm kiếm tĩnh trên repo, không phụ thuộc Isaac Sim GUI.

```
# Chạy tại thư mục repo baseline
mkdir -p outputs
```

```
find . -type f \( -name "*.py" -o -name "*.yaml" -o -name "*.yml" -o -name "*.json" -o -name "*.urdf"
\) \
| grep -v ".git" \
| xargs grep -nEi
"pose|transform|quaternion|euler|rotation|ik|jacobian|joint|link|camera|gripper|end.?effector" \
> outputs/baseline_math_map_raw.txt
```

```
python tools/summarize_math_map.py outputs/baseline_math_map_raw.txt outputs/baseline_math_map.md
```

Script gợi ý tools/summarize_math_map.py

```
from pathlib import Path
import sys

inp = Path(sys.argv[1])
out = Path(sys.argv[2])

keywords = ["pose", "transform", "quaternion", "rotation", "ik", "jacobian", "joint", "camera",
"grripper"]
counts = {k: 0 for k in keywords}
examples = {k: [] for k in keywords}
for line in inp.read_text(encoding="utf-8", errors="ignore").splitlines():
    low = line.lower()
    for k in keywords:
        if k in low:
            counts[k] += 1
            if len(examples[k]) < 5:
                examples[k].append(line[:220])

with out.open("w", encoding="utf-8") as f:
    f.write("# Baseline Math Map\n\n")
    for k in keywords:
        f.write(f"## {k} ({counts[k]} hits)\n")
        for e in examples[k]:
            f.write(f"- {e}\n")
        f.write("\n")
print(f"Wrote {out}")
```

Điều kiện pass	Ngưỡng tối thiểu
File baseline_math_map.md tồn tại	Có ít nhất 6 nhóm keyword và mỗi nhóm có ví dụ file:line.
Sinh viên giải thích được 3 file quan trọng nhất	Mỗi file ghi: chức năng, input/output, liên quan đến frame/pose/IK như thế nào.
Không kết luận mơ hồ	Không được viết “em thấy có transform trong code” mà không chỉ đường dẫn file và dòng liên quan.

18.2. Demo 1 - Inspect URDF: joint/link/end-effector inventory

Ý nghĩa. URDF cho biết robot có những link/joint nào. Dù sinh viên chưa điều khiển robot thật, việc đọc URDF giúp hiểu FK là chuỗi biến đổi qua các joint, còn IK là tìm joint values để đạt pose mong muốn.

```
# Ví dụ tìm file URDF sau khi tải assets
find . -type f -iname "*.urdf" -o -iname "*.xacro"
```

```
# Chạy parser
python tools/inspect_urdf_joints.py --urdf path/to/WalkerS2.urdf --out outputs/joint_inventory.csv
```

Script gợi ý tools/inspect_urdf_joints.py

```
import argparse, csv, xml.etree.ElementTree as ET
from pathlib import Path

parser = argparse.ArgumentParser()
parser.add_argument("--urdf", required=True)
parser.add_argument("--out", default="outputs/joint_inventory.csv")
args = parser.parse_args()
root = ET.parse(args.urdf).getroot()
rows = []
```

```

for j in root.findall("joint"):
    name = j.attrib.get("name", "")
    jtype = j.attrib.get("type", "")
    parent = j.find("parent").attrib.get("link", "") if j.find("parent") is not None else ""
    child = j.find("child").attrib.get("link", "") if j.find("child") is not None else ""
    axis = j.find("axis").attrib.get("xyz", "") if j.find("axis") is not None else ""
    limit = j.find("limit")
    lower = limit.attrib.get("lower", "") if limit is not None else ""
    upper = limit.attrib.get("upper", "") if limit is not None else ""
    rows.append([name, jtype, parent, child, axis, lower, upper])
Path(args.out).parent.mkdir(parents=True, exist_ok=True)
with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["joint", "type", "parent", "child", "axis_xyz", "lower", "upper"])
    w.writerows(rows)
print(f"Wrote {args.out} with {len(rows)} joints")

```

Câu hỏi kiểm tra sau demo	Trả lời trong báo cáo
Joint nào thuộc tay trái/tay phải/waist?	Liệt kê theo tên joint hoặc theo parent-child chain nếu naming không rõ.
End-effector hoặc gripper link có thể là link nào?	Đưa ra giả thuyết và bằng chứng từ tên link, child chain, mesh hoặc config.
Joint limit ảnh hưởng IK thế nào?	Giải thích target dù trong không gian 3D vẫn có thể không đạt nếu joint limit/singularity.

18.3. Demo 2 - Frame/config inventory

Ý nghĩa. Trong HRC2026, lỗi thường gặp là nhầm camera/world/robot base/object frame. Demo này yêu cầu sinh viên tạo một bảng inventory các frame có thể tìm thấy từ config, URDF, USD path hoặc log baseline.

```

mkdir -p outputs
python tools/scan_frame_inventory.py --root . --out outputs/frame_inventory.csv
# tools/scan_frame_inventory.py - bản tối giản, quét theo keyword trong file text
import argparse, csv
from pathlib import Path
parser = argparse.ArgumentParser()
parser.add_argument("--root", default=".")
parser.add_argument("--out", default="outputs/frame_inventory.csv")
args = parser.parse_args()
keywords = ["world", "base", "camera", "wrist", "gripper", "ee", "end_effector", "bin", "foam",
"carton", "conveyor"]
exts = {".py", ".yaml", ".yml", ".json", ".urdf", ".usd", ".usda"}
rows = []
for path in Path(args.root).rglob("*"):
    if path.is_file() and path.suffix.lower() in exts and ".git" not in path.parts:
        try:
            text = path.read_text(encoding="utf-8", errors="ignore")
        except Exception:
            continue
        low = text.lower()
        hits = [k for k in keywords if k in low]
        if hits:
            rows.append([str(path), ";".join(hits)])
Path(args.out).parent.mkdir(parents=True, exist_ok=True)
with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["file", "frame_keywords"]); w.writerows(rows)
print(f"Wrote {args.out} with {len(rows)} candidate files")

```

18.4. Demo 3 - Transform sanity trong Python

Ý nghĩa. Demo này dùng cùng công thức đã học ở các mục SO(3), quaternion và SE(3) của bản gốc. Mục tiêu là tạo một bài kiểm thử nhanh trước khi dùng pose trong controller.

```
python tools/transform_sanity.py --out outputs/transform_sanity_report.json
```

```
import json, numpy as np
```

```
from scipy.spatial.transform import Rotation as R
```

```
def make_T(rot, t):
```

```
    T = np.eye(4)
```

```
    T[:3, :3] = rot.as_matrix()
```

```
    T[:3, 3] = np.array(t, dtype=float)
```

```
    return T
```

```
rot = R.from_euler("xyz", [20, -10, 45], degrees=True)
```

```
T = make_T(rot, [0.3, -0.2, 0.5])
```

```
I_err = float(np.linalg.norm(T @ np.linalg.inv(T) - np.eye(4)))
```

```
q_xyzw = rot.as_quat()
```

```
R2 = R.from_quat(q_xyzw).as_matrix()
```

```
roundtrip_err = float(np.linalg.norm(R2 - rot.as_matrix()))
```

```
report = {"inverse_error": I_err, "quat_roundtrip_error": roundtrip_err, "pass": I_err < 1e-9 and  
roundtrip_err < 1e-9}
```

```
open("outputs/transform_sanity_report.json", "w", encoding="utf-8").write(json.dumps(report,  
indent=2))
```

```
print(report)
```

18.5. Demo 4 - IK dry-run với target pose an toàn

Ý nghĩa. Demo này không yêu cầu sinh viên tự viết full IK cho Walker S2. Sinh viên chỉ cần biết cách gọi hoặc tìm IK API trong baseline, gửi target pose an toàn, và đọc residual/trạng thái trả về. Nếu môi trường baseline chưa sẵn sàng, dùng IK 2R trong Lab 4 làm fallback bắt buộc.

Pseudo-code. Tên class/function cần điều chỉnh theo baseline thực tế sau khi đọc repo.

```
from hrc_baseline_api import load_robot, load_ik_solver
```

```
robot = load_robot(config="configs/task1.yaml", headless=True)
```

```
ik = load_ik_solver(robot, arm="right")
```

```
target_pose_base = {
```

```
    "position": [0.45, -0.20, 0.35],
```

```
    "orientation_xyzw": [0.0, 0.7071, 0.0, 0.7071]
```

```
}
```

```
q_sol, info = ik.solve(target_pose_base, seed="home")
```

```
print(info) # expected: success, residual_position, residual_orientation, joint_limit_violation
```

Output	Nội dung bắt buộc
ik_dryrun_report.csv	target_id, arm, x, y, z, qx, qy, qz, qw, success, pos_residual, ori_residual, joint_limit_violation, note
Ảnh/log nếu chạy được Isaac Sim	Một frame trước/sau hoặc log end-effector pose. Nếu không chạy được GUI, phải có headless log hoặc giải thích lý do.
Fallback	Nếu baseline IK chưa chạy, nộp ik_2r_report.csv và baseline_math_map.md chỉ ra API IK cần đọc tiếp.

19. Lab bổ sung có output đo được

Cách dùng. Các lab trong bản gốc vẫn giữ nguyên. Phần dưới đây bổ sung yêu cầu đo lường để mọi lab có sản phẩm chấm được. Sinh viên đặt toàn bộ file trong thư mục module3_outputs/ và commit vào branch cá nhân hoặc nộp zip theo yêu cầu của giảng viên.

Lab	Mục tiêu	Output bắt buộc	Ngưỡng pass
Lab 1 - Rotation convention	Kiểm tra SO(3), Euler, quaternion, rotnec.	rotation_report.csv, pytest_report.txt	orthogonality_error < 1e-6; det_error < 1e-6; quat_roundtrip_error < 1e-6.
Lab 2 - SE(3) transform chain	Ghép T_world_camera, T_camera_object, T_base_world để tính pose object trong base.	transform_chain_report.json, tests/test_transforms.py	inverse_error < 1e-6; composition_error < 1e-6; giải thích đúng chiều T A B.
Lab 3 - Camera-to-base toy example	Từ pixel/depth giả lập tính point 3D trong base frame.	camera_to_base_example.csv, camera_to_base_explanation.md	Sai số so với đáp án mẫu < 1e-6 nếu dùng input mẫu.
Lab 4 - FK/IK 2R planar arm	Viết FK, IK analytic, IK numerical cho tay 2 khớp.	ik_2r_report.csv, ik_2r_workspace.png	mean residual < 1e-3 m cho target reachable; phát hiện đúng target unreachable.

Lab 5 - Workspace map	Vẽ vùng với tới được và đánh dấu singularity/joint limits.	workspace.png, workspace_summary.md	Có ít nhất 1000 samples; hình có trục, title, chú giải.
Lab 6 - Baseline math map	Tìm math concepts trong baseline/assets.	baseline_math_map.md, joint_inventory.csv, frame_inventory.csv	Có đường dẫn file cụ thể và 3 nhận xét liên hệ với HRC2026.

19.1. Lab 1 - Rotation convention audit

- Input: 10 bộ Euler angle tự chọn, gồm cả góc nhỏ, góc lớn, và ít nhất 1 trường hợp gần gimbal lock.
- Xử lý: chuyển Euler $\rightarrow$ rotation matrix $\rightarrow$ quaternion $\rightarrow$ rotation matrix; kiểm tra $R^T R = I$ và $\det(R) = 1$.
- Output: rotation_report.csv có các cột sample_id, roll, pitch, yaw, qx, qy, qz, qw, orthogonality_error, det_error, roundtrip_error, pass.

```
# Lệnh nộp bài gợi ý
python labs/lab1_rotation_audit.py --out module3_outputs/rotation_report.csv
pytest tests/test_rotation.py > module3_outputs/pytest_report.txt
```

19.2. Lab 2 - SE(3) transform chain

Bài toán mẫu. Cho $T_{\text{world_camera}}$, $T_{\text{camera_object}}$ và $T_{\text{base_world}}$. Hãy tính $T_{\text{base_object}}$. Sau đó kiểm tra bằng cách biến đổi một point object-local sang base frame theo hai cách khác nhau và so sánh sai số.

```
# Công thức kỳ vọng
T_base_object = T_base_world @ T_world_camera @ T_camera_object
p_base = T_base_object @ p_object_h
# Sanity check bắt buộc
assert np.linalg.norm(T_base_object @ np.linalg.inv(T_base_object) - np.eye(4)) < 1e-6
```

File	Nội dung
transform_chain_report.json	input transforms, output $T_{\text{base_object}}$, inverse_error, composition_error, pass.
transform_chain_explanation.md	Giải thích bằng lời vì sao thứ tự nhân ma trận là như vậy.

19.3. Lab 3 - Camera-to-base toy example

Mục đích. Chuẩn bị cho Module 4. Sinh viên chưa cần xử lý ảnh thật, chỉ cần dùng công thức pinhole camera và transform đã học để đổi một pixel/depth giả lập sang robot base frame.

```
# Input mẫu
u, v, z = 410, 260, 0.75
fx, fy, cx, cy = 600, 600, 320, 240
X_cam = (u - cx) * z / fx
Y_cam = (v - cy) * z / fy
Z_cam = z
p_cam = np.array([X_cam, Y_cam, Z_cam, 1.0])
p_base = T_base_camera @ p_cam
```

- Output bắt buộc: camera_to_base_example.csv gồm u, v, z, X_{cam} , Y_{cam} , Z_{cam} , X_{base} , Y_{base} , Z_{base} .
- Báo cáo phải ghi rõ $T_{\text{base_camera}}$ dùng trong ví dụ và đơn vị mét/radian.

19.4. Lab 4 - FK/IK 2R planar arm có residual

Mục đích. Trước khi xử lý robot nhiều khớp, sinh viên cần chứng minh hiểu FK/IK bằng tay máy 2 khớp. Đây là bài lab bắt buộc, có output định lượng.

```
# Công thức FK 2R
x = L1*np.cos(q1) + L2*np.cos(q1 + q2)
y = L1*np.sin(q1) + L2*np.sin(q1 + q2)
```

Cột trong ik_2r_report.csv	Ý nghĩa
target_id, x_target, y_target	Target cần với tới.
reachable	True/False dựa trên khoảng cách so với $L_1 + L_2$ và $ L_1 - L_2 $.
q1_solution, q2_solution	Một nghiệm IK được chọn.
x_fk, y_fk	Vị trí tính lại bằng FK từ nghiệm IK.
residual	$\sqrt{(x_{\text{fk}} - x_{\text{target}})^2 + (y_{\text{fk}} - y_{\text{target}})^2}$.
status	success, unreachable, singular, joint_limit_violation.

19.5. Liên hệ output lab với 4 task HRC2026

Task	Khái niệm Module 3 cần dùng	Lab/output liên quan
Task 1 - Desktop sorting	Pose vật trên bàn, bin frame, pre-grasp pose, place pose.	transform_chain_report.json, rotation_report.csv, baseline_math_map.md.
Task 2 - Conveyor sorting	Pose theo thời gian, extrapolation trong frame robot/base, target intercept.	camera_to_base_example.csv, transform_chain_report.json.
Task 3 - Workpiece insertion	Slot frame, orientation alignment, downward insertion direction, residual pose error.	rotation_report.csv, transform_chain_report.json, IK residual report.
Task 4 - Carton sealing	Hai end-effector frames, relative pose giữa hai tay, sequence pose targets.	joint_inventory.csv, frame_inventory.csv, IK dry-run report.

20. Rubric cập nhật và gói nộp Module 3

Rubric này bổ sung cho rubric mini-project đã có trong bản gốc. Nếu giảng viên muốn chấm nhanh, có thể dùng thang 100 điểm dưới đây; trong đó 60 điểm dành cho lý thuyết/code gốc, 25 điểm cho baseline demo, 15 điểm cho reproducibility/report.

Hạng mục	Điểm	Yêu cầu đạt điểm tối đa
Giữ đúng lý thuyết và convention	20	Giải thích đúng point/vector/pose/frame, $SO(3)$, quaternion convention, $SE(3)$, T A B, FK/IK. Không dùng biến mơ hồ thiếu frame.
Unit tests toán học	15	rotation/transform tests pass; có báo cáo sai số và ngưỡng pass rõ ràng.
FK/IK mini lab	15	IK 2R chạy được, residual thấp, phát hiện unreachable/singularity/joint-limit cases.
Workspace và collision intuition	10	Có workspace.png, phân tích vùng không với tới được và rủi ro va chạm ở mức khái niệm.
Demo baseline math map	15	Có baseline_math_map.md, chỉ rõ file/function liên quan pose/transform/IK/camera/gripper.
URDF/frame inventory	10	Có joint_inventory.csv và frame_inventory.csv hoặc giải thích fallback nếu asset chưa tải được.
Báo cáo và reproducibility	15	README có lệnh chạy, môi trường, output list, giải thích lỗi gặp phải và cách debug. Clone sạch chạy lại được.

20.1. Cấu trúc gói nộp khuyến nghị

```

module3_submission_<student_id>/
  README.md
  requirements.txt
  src/
    robot_math.py
    ik_2r.py
    scan_baseline_math.py
    inspect_urdf_joints.py
  tests/
    test_rotation.py
    test_transforms.py
    test_ik_2r.py
  module3_outputs/
    pytest_report.txt
    rotation_report.csv
    transform_chain_report.json
    camera_to_base_example.csv
    ik_2r_report.csv
    workspace.png
    baseline_math_map.md
    joint_inventory.csv
    frame_inventory.csv
    baseline_demo_report.md

```

20.2. Template README nộp bài

```

# Module 3 - Robotics Math Submission

## 1. Environment
- Python version:

```

- Packages:
- Baseline repo path or commit:

2. Frame convention

- T_A_B means:
- Quaternion order used in this submission:
- Units:

3. How to run

```
```bash
pip install -r requirements.txt
pytest tests/ > module3_outputs/pytest_report.txt
python src/lab1_rotation_audit.py
python src/lab2_transform_chain.py
python src/ik_2r.py
python src/scan_baseline_math.py --root /path/to/baseline
```
```

4. Output summary

| Output file | Status | Notes |
|-----------------------------|-----------|-------|
| rotation_report.csv | pass/fail | |
| transform_chain_report.json | pass/fail | |
| ik_2r_report.csv | pass/fail | |
| workspace.png | pass/fail | |
| baseline_math_map.md | pass/fail | |

5. What I learned and bugs found

- Bug 1:
- Bug 2:
- Next step for HRC2026 task:

20.3. Tài liệu tham khảo bổ sung cho phần demo/lab

| Tài liệu | Link | Dùng để làm gì |
|---|---|---|
| UBTECH GlobalHumanoidRobotChallenge_2026_Baseline | https://github.com/UBTECH-Robot/GlobalHumanoidRobotChallenge_2026_Baseline | Repo baseline chính thức; dùng để tìm file pose/transform/I K, đọc config và chạy demo. |
| UBTECH challenge2026_assets | https://huggingface.co/UBTECH-Robotics/challenge2026_assets | Assets Walker S2, scene, vật thể, conveyor, foam/carton; dùng cho URDF/USD/frame inventory. |
| UBTECH WalkerS2-Model | https://github.com/UBTECH-Robot/WalkerS2-Model | Model URDF/USD của Walker S2; dùng để đọc link/joint/kinematic chain. |
| Modern Robotics - Chapters 3-6 | https://modernrobotics.northwestern.edu/chapters/chapter3/ | Nền tảng rigid-body motion, SE(3), FK, Jacobian, IK. |
| Modern Robotics Code Library | https://github.com/NxRLab/ModernRobotics | Code Python/MATLAB cho SE(3), FK/IK, Jacobian. |
| SciPy Rotation | https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html | API chuyển đổi rotation matrix, Euler, quaternion, rotvec. |
| ROS 2 tf2 Concepts | https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html | Cách tư duy transform tree, |

HRC2026 Training Camp | Module 3: Robotics Math

| | | |
|--------------------------------|---|---|
| | | frame, lookup transform. |
| NVIDIA Isaac Sim Documentation | https://docs.isaacsim.omniverse.nvidia.com/ | Tra cứu stage, articulation, robot simulation, camera/sensor, Python scripting. |
| Robotics Toolbox for Python | https://petercorke.github.io/robotics-toolbox-python/ | Công cụ học FK/IK, trajectory, robot model ở mức học thuật. |
| SpatialMath Python | https://bdaiinstitute.github.io/spatialmath-python/ | Công cụ biểu diễn $SE(3)$, $SO(3)$, quaternion, transform trong Python. |